URBANMART RETAIL

Sales Analytics - MySQL Project Report

Prepared by: Yash Govindwar | Tool: MySQL 8.x / MySQL Workbench | Source period: 2025

End-to-end SQL analysis of 2025 retail sales, customer value, store performance and inventory signals

6,000	₹1.75 Cr	350	90	9
sales transactions	net revenue	customers	SKUs	store cities

Executive Summary

UrbanMart is a fictional Indian retail chain operating across 9 cities and 6 product categories. The project uses two linked MySQL tables - a 90-SKU product master and a 6,000-row transaction table - covering 1 Jan to 28 Dec 2025. The analysis focuses on revenue, customer behaviors, store performance, payment mix and inventory health, using MySQL from basic filtering through joins, aggregation, subqueries, CTEs and window functions.

- Electronics generated the highest revenue at ₹0.59 Cr, contributing 33.9% of total revenue.
- Pune was the top store city at ₹0.36 Cr and Pune + Mumbai together contributed 40.4% of annual revenue; December was the peak month at ₹0.25 Cr.
- UPI was the most-used payment mode, accounting for 39.8% of net revenue.
- One SKU, P1029 - Besan 1kg, was below its reorder level and had 217 units sold, making it the clearest restock signal in the supplied data.

1. Business Problem & Data Architecture

UrbanMart wants a single analytical view of retail performance. The project specification focuses on revenue trends, customer buying behavior , store performance and inventory health, using SQL queries that progress from filtering to joins, aggregation, subqueries, CTEs and window functions.

PRODUCTS
90 SKUs

product_id (PK)
category | brand
cost_price | selling_price
stock_quantity | reorder_level
supplier_city

SALES_TRANSACTIONS
6,000 line-item sales

transaction_id (PK)
product_id (FK)
sale_date | customer_id
store_city | store_location
quantity | discount_pct
net_amount | payment_mode

Key Data Model

Area	Implementation in supplied project
Database	urbanmart_retail
Relationship	products.product_id -> sales_transactions.product_id
Performance	Indexes on sale_date, customer_id and store_city
Loading	MySQL Workbench Import Wizard or LOAD DATA LOCAL INFILE
Validation target	90 products; 6,000 transactions; 2025-01-01 to 2025-12-28

Analytical Findings Covered, With Query Number referred to Sql file.

Basic Queries

1. List of all products that belong to the 'Electronics' category.
2. All transactions made using the 'UPI' payment mode.
3. List of cities where UrbanMart has store locations (store_city).
4. List of all products priced above ₹2,000, sorted by selling price high to low.
5. All transactions where the discount given was greater than 15%.
6. Product name, brand and category for products supplied from 'Pune'.
7. Total number of transactions recorded in the sales_transactions table.

Intermediate Queries

1. Total revenue (net_amount) generated by each product category.
2. Top 5 customers by total amount spent, along with their city.
3. Average discount percentage given per store city.
4. Number of transactions and total quantity sold for each payment mode.
5. All products that have never appeared in any sales transaction (products with zero sales).
6. Month-wise total sales revenue for the year 2025.

7. Categories whose total revenue exceeds ₹5,00,000.
8. For each store city, single highest-value transaction (by net amount).
9. .Number of unique customers who made a purchase from each store city.
10. Total revenue generated by the Electronics category.
11. Total revenue generated through UPI payments.
12. Highest-revenue month of 2025.

Advanced Queries

1. Ranking products within each category by total revenue generated.
2. List of top selling products in each city.
3. The running (cumulative) monthly revenue for 2025 using a window function.
4. Customers whose total spend is higher than the average total spend across all customers.
5. Month-over-month revenue growth percentage using.
6. Products where current stock quantity is below reorder level and ranking them by restock priority based on how frequently they have sold.
7. Segment customers into 'High', 'Medium' and 'Low' value tiers based on total spend and count of customers per tier.
8. Comparison of average monthly revenue during the festive period (October–December) against the rest of the year, for each store city.
9. Write a query using a CTE that returns, for every category, the best-selling month of the year by revenue.
10. Percentage of total revenue contributed by Pune and Mumbai together.
11. List of products below their reorder level and rank them by sales velocity.

The schema and loading scripts explicitly define the primary/foreign-key relationship and the expected data volumes.

2. SQL Analysis Approach

The solution file is organised into Beginner, Intermediate and Advanced sections. This matches the source project structure and demonstrates a progression from basic retrieval to analytical SQL patterns.

Level	Techniques used	Business output
Beginner	SELECT, WHERE, ORDER BY, DISTINCT, COUNT	Product filtering, UPI sales, store cities, price/discount checks
Intermediate	JOIN, GROUP BY, HAVING, MAX, AVG	Category revenue, customer spend, payment mix, monthly revenue
Advanced	CTE, RANK/DENSE_RANK, ROW_NUMBER, LAG, CASE	Product rankings, city leaders, MoM growth, stock priority, customer tiers

Selected Query Logic

- Category revenue joins products to sales_transactions on product_id and aggregates net_amount by category.
- Top-selling product by city uses a CTE plus ROW_NUMBER() partitioned by store_city.
- Monthly cumulative revenue uses SUM(SUM(net_amount)) OVER (ORDER BY month).
- MoM growth uses LAG() to compare the current month with the prior month.
- Restock priority joins product stock levels to aggregated sold quantity and ranks the below-reorder items.

Portfolio-Level SQL Evidence

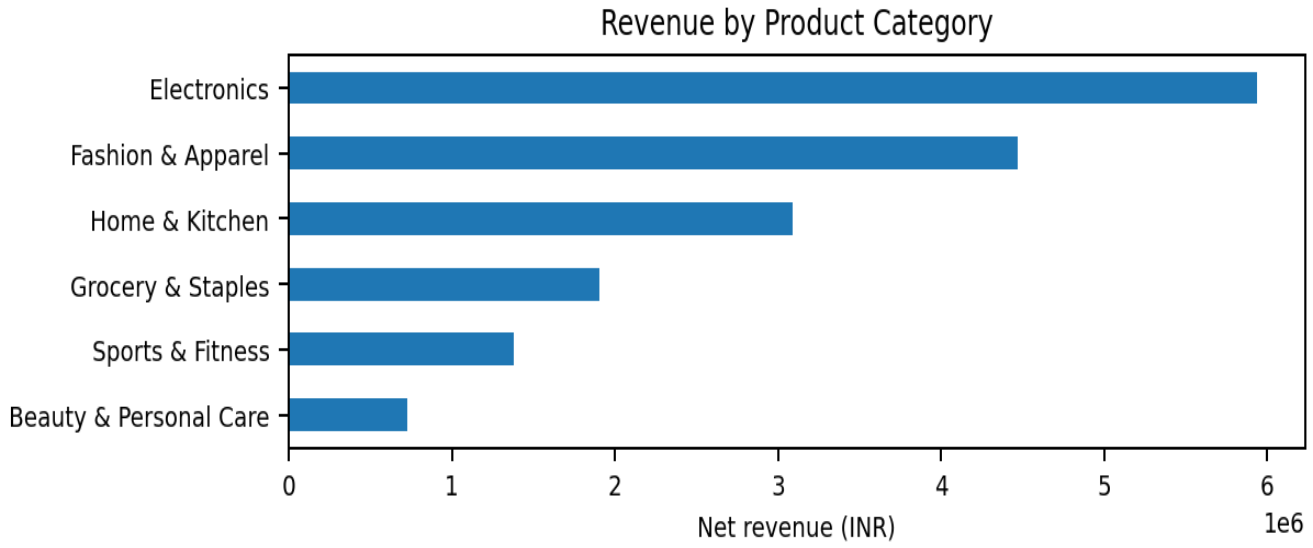
6 product categories	**25+** analysis tasks in the guide	**3** SQL levels	**1** relational product-sales link

The project guide frames the work as a resume-ready SQL project covering 6,000+ transactions, 90 SKUs, 9 cities and 6 categories, with joins, aggregations, subqueries, CTEs and window functions. The added findings above translate those SQL outputs into concise business actions.

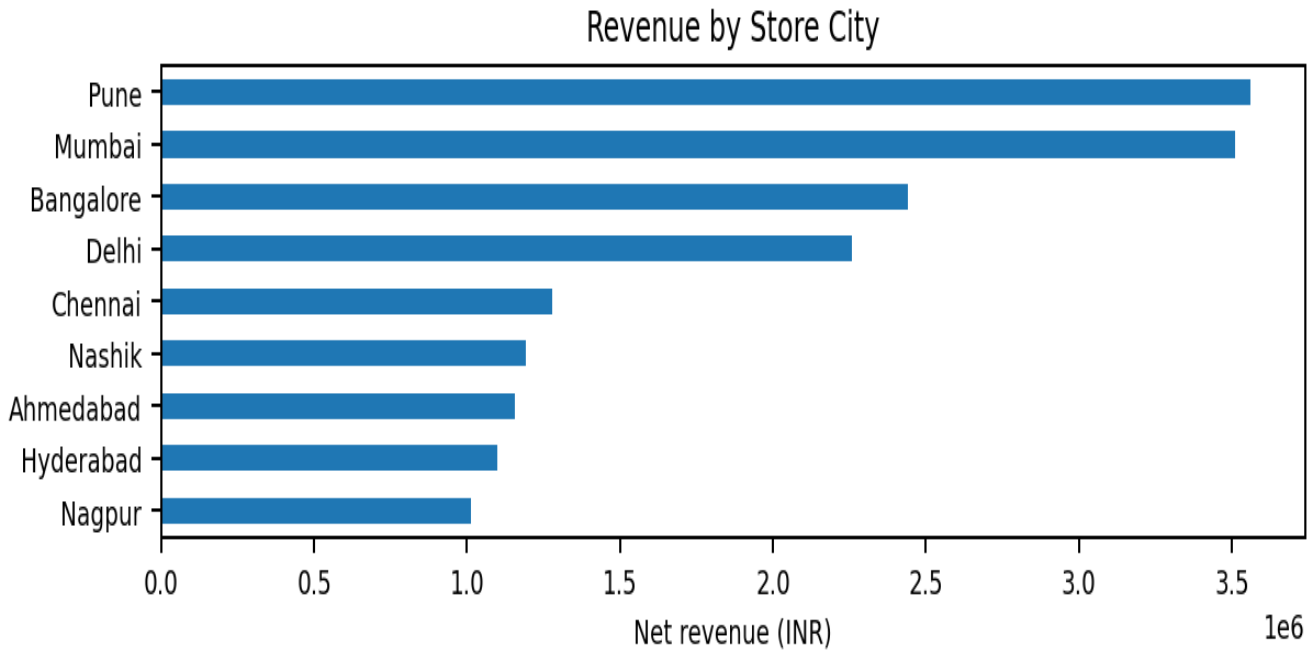
Important Result Snapshot - Five additional quantified findings

Metric	Value	Interpretation
Electronics	₹5,935,613	33.9% of revenue; #1 category
Pune + Mumbai	₹7,073,880	40.4% of annual revenue
UPI	₹6,969,902	39.8% of net revenue; #1 payment mode
December	₹2,507,730	Peak month; 39.4% above November
P1029 - Besan 1kg	16 vs 20 units	Only SKU below reorder level; 217 units sold

3. Business Findings - Revenue & Store Performance



Electronics leads with ₹5,935,613 (33.9% of revenue), followed by Fashion & Apparel at ₹4,468,450. Together, the two categories contribute 59.4% of annual revenue. Among individual SKUs, Smartwatch is the top product at ₹540,748.



Pune is the highest-revenue store city at ₹3,561,704, narrowly ahead of Mumbai at ₹3,512,177. Together they contribute approximately 40.4% of annual revenue, highlighting two priority markets for inventory and customer-growth initiatives.

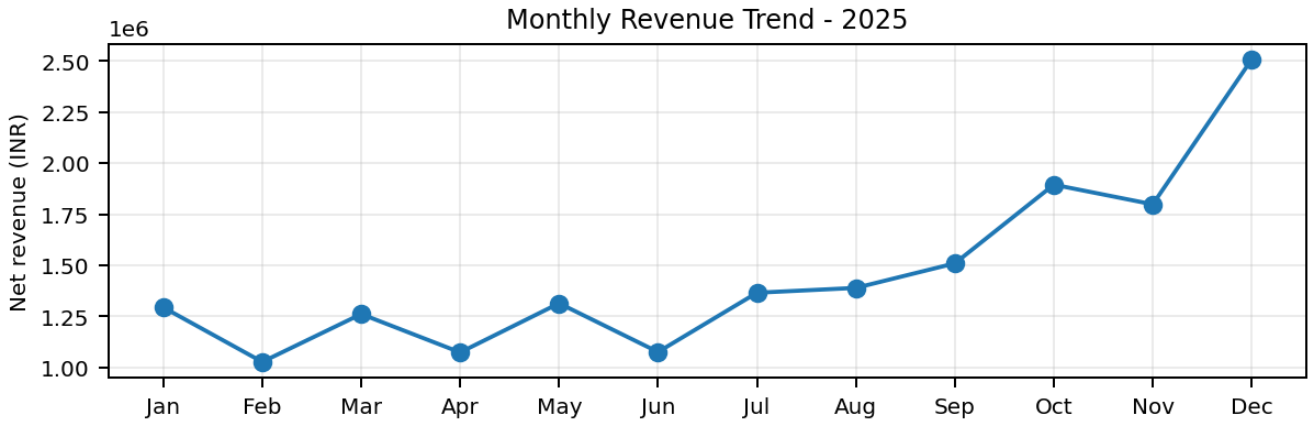
DECEMBER EFFECT

December revenue was ₹2,507,730, 39.4% above November. Oct-Dec contributed 35.4% of annual revenue.

PAYMENT MIX

UPI generated ₹6,969,902, 39.8% of net revenue and was the leading payment mode.

4. Customer, Payment & Inventory Insights



The monthly pattern is relatively steady through most of the year, followed by a strong Q4 uplift. December is the highest month at ₹2,507,730, up 39.4% from November; October-December contributes 35.4% of annual revenue. UPI is also the leading payment mode at ₹6,969,902, or 39.8% of net revenue.

Customer Value Segmentation

Tier	Rule from solution	Customers
Low	Total spend <= ₹30,000	33
Medium	₹30,001-₹69,999	277
High	>= ₹70,000	40

Top 5 Products by Net Revenue

Product	Category	Revenue
Smartwatch	Electronics	₹540,748
Extension Board 4-Socket	Electronics	₹535,768
USB Type-C Cable	Electronics	₹534,727
Portable Charger 20000mAh	Electronics	₹510,035
Mechanical Keyboard	Electronics	₹498,160

Inventory Signal

One SKU, P1029 - Besan 1kg, was below its reorder level (16 vs 20 units) and had 217 units sold, making it the clearest restock signal in the supplied data.

These findings support targeted actions: protect availability of fast-moving low-stock items, prioritize Pune and Mumbai, strengthen Electronics availability, optimize UPI-led offers, and plan inventory/promotions ahead of the Q4 demand spike.

5. Recommendations, Quality Review.

Business Recommendations

- Prioritise replenishment of below-reorder, high-velocity SKUs; the current dataset flags P1029 as the immediate restock action.
- Use Electronics and Fashion & Apparel as core revenue levers while monitoring category concentration and protecting the top Electronics SKUs.
- Plan promotions and inventory capacity ahead of Q4, especially December, which is the strongest month and contributes to a 35.4% Q4 revenue share.
- Design retention campaigns around the High-value tier, localise offers for Pune and Mumbai, and consider UPI-focused promotions because UPI accounts for 39.8% of revenue.

SQL Implementation Quality Review

Type	Observation
Strength	Broad SQL coverage: joins, aggregation, subqueries, CTEs, ranking and LAG/LEAD.
Issue to fix	The zero-sales query uses `= NULL`; SQL requires `IS NULL`.
Issue to fix	The top-5-customer query groups by store_city, which can split one customer across cities.
Issue to fix	The "above average customer spend" query compares one transaction amount with average transaction amount, not customer-level totals.
Data-model note	The project files use both `sales_transaction` and `sales_transactions`; standardise the table name in the final portfolio version.

Project Summary

UrbanMart Retail Sales Analytics - MySQL: Designed and queried a two-table retail schema covering 6,000 transactions across 90 SKUs, 9 cities and 6 categories; used joins, aggregations, subqueries, CTEs and window functions to analyse revenue trends, customer value, store performance and inventory restock priorities.

Source basis: UrbanMart project specification, database/schema scripts, solution SQL, products.csv and sales_transactions.csv supplied . Calculations in this report were recomputed directly from the supplied CSV files.